

Especificación de implementación: conversión de Ticketera CC en una plataforma genérica multi-organización

1. Objetivo

Modificar la aplicación existente para convertirla de una ticketera específica de una organización en una plataforma genérica de ticketing utilizada por múltiples organizaciones independientes.

La plataforma debe permitir que:

1. múltiples organizaciones utilicen una misma instancia de la aplicación;
2. cada organización administre sus propios eventos;
3. usuarios administradores de una organización creen y editen eventos de esa organización;
4. los eventos publicados sean visibles públicamente sin autenticación;
5. un usuario público pueda registrarse o iniciar sesión para comprar tickets;
6. una compra confirmada genere tickets asociados al usuario;
7. los tickets sean enviados al comprador por email;
8. los datos y permisos de cada organización permanezcan aislados de las demás organizaciones.

La implementación debe evolucionar el código existente y reutilizar sus mecanismos funcionales siempre que sea razonable. No realizar una reescritura completa.

2. Repositorio objetivo

Repositorio:

`https://github.com/ceciliamaas/ticketera-cc`

Trabajar sobre la rama de desarrollo correspondiente al flujo actual del proyecto.

Antes de modificar código:

1. inspeccionar el repositorio completo;
2. identificar los modelos existentes de eventos, tickets, usuarios, pagos, caja y otras funcionalidades;
3. identificar las vistas, formularios, templates, URLs, señales, tareas programadas y servicios que dependen de esos modelos;
4. identificar el flujo actual de MercadoPago y sus webhooks;
5. identificar el flujo actual de emisión y envío de tickets;
6. identificar tests existentes;
7. identificar código específico de Fuego Austral que afecte el dominio central;
8. elaborar internamente un mapa de dependencias antes de comenzar los cambios.

No asumir que los nombres de modelos o campos propuestos en este documento coinciden con el código actual. Adaptar la implementación a la estructura real cuando sea más seguro y simple.

3. Principios obligatorios de implementación

3.1 Evolución incremental

No reemplazar la aplicación existente por una arquitectura nueva desde cero.

Priorizar:

- reutilización de modelos existentes;
- migraciones incrementales;
- compatibilidad con datos existentes;
- preservación de flujos funcionales;
- cambios pequeños y verificables.

3.2 Organización como límite de propiedad y autorización

`Organization` debe ser una entidad explícita del dominio.

Todo recurso organizacional debe tener una relación inequívoca con una organización.

Como mínimo:

```
Organization
└─ Event
```

Y todo dato administrativo derivado de eventos debe poder restringirse a la organización correspondiente.

3.3 Usuario global, permisos organizacionales

No crear sistemas de identidad independientes para compradores y administradores.

Debe existir un único usuario global.

Un mismo usuario puede:

- ser comprador;
- ser administrador de una organización;
- ser administrador de varias organizaciones;
- tener roles diferentes en organizaciones diferentes.

3.4 Autorización en backend

Nunca confiar únicamente en:

- botones ocultos;
- navegación;
- templates;
- JavaScript;
- URLs no enlazadas.

Toda lectura o mutación administrativa debe validar permisos y pertenencia organizacional en backend.

3.5 Compatibilidad con datos existentes

Los eventos, tickets, compras y usuarios existentes no deben perderse.

Las migraciones deben ser:

- reproducibles;
- revisables;
- seguras;
- compatibles con despliegue incremental.

No eliminar columnas ni modelos existentes durante la primera fase si contienen datos o si todavía existen dependencias activas.

3.6 No sobrearquitecturar

No introducir inicialmente:

- una base de datos por organización;
- un schema PostgreSQL por organización;
- microservicios;
- una SPA;
- un nuevo framework frontend;
- un sistema externo de autorización;
- un event bus;
- una reescritura del sistema de pagos.

Usar la arquitectura Django existente.

4. Alcance funcional obligatorio

4.1 Organizaciones

La plataforma debe soportar múltiples organizaciones.

Cada organización debe tener como mínimo:

- nombre;
- slug único;
- estado activo/inactivo;
- timestamps de creación y modificación.

Modelo conceptual:

```
class Organization(models.Model):
    name = models.CharField(max_length=255)
    slug = models.SlugField(unique=True)
    is_active = models.BooleanField(default=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

El agente debe adaptar nombres, estilo y convenciones al proyecto.

Requisitos

- El `slug` debe ser único.
- Una organización inactiva no debe poder publicar nuevos eventos ni operar normalmente.
- Los eventos de una organización inactiva no deben mostrarse en el catálogo público salvo que exista una razón funcional explícita y documentada.
- `Organization` debe registrarse en Django Admin.

5. Membresías y roles organizacionales

Crear una relación explícita entre usuarios y organizaciones.

Modelo conceptual:

```
class OrganizationMembership(models.Model):
    class Role(models.TextChoices):
        OWNER = "owner", "Owner"
        ADMIN = "admin", "Admin"
        EDITOR = "editor", "Editor"

    organization = models.ForeignKey(
        Organization,
        on_delete=models.CASCADE,
        related_name="memberships",
    )

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
```

```

        on_delete=models.CASCADE,
        related_name="organization_memberships",
    )

    role = models.CharField(
        max_length=20,
        choices=Role.choices,
    )

```

Agregar una restricción única:

```
organization + user
```

Un usuario no puede tener dos memberships diferentes para la misma organización.

5.1 Roles iniciales

Owner

Puede:

- administrar la organización;
- administrar miembros;
- crear eventos;
- editar eventos;
- publicar eventos;
- ver datos administrativos de la organización.

Admin

Puede:

- crear eventos;
- editar eventos;
- publicar eventos;
- ver datos administrativos de la organización.

No asumir automáticamente que puede:

- eliminar la organización;
- transferir ownership.

Editor

Puede:

- crear eventos;
- editar eventos.

La capacidad de publicar puede dejarse inicialmente restringida a `owner` y `admin`.

5.2 Platform superuser

Los usuarios Django con privilegios globales apropiados deben conservar capacidad de administración global.

No sustituir el mecanismo de superusuario de Django por memberships organizacionales.

5.3 Helpers de permisos

Centralizar la lógica.

Ejemplos conceptuales:

```
user_is_organization_member(user, organization)
user_can_manage_organization(user, organization)
user_can_create_event(user, organization)
user_can_edit_event(user, event)
user_can_publish_event(user, event)
```

No dispersar comparaciones manuales de strings de roles por decenas de vistas.

Preferir una capa coherente, por ejemplo:

```
organizations/permissions.py
```

o la convención equivalente ya utilizada en el proyecto.

6. Eventos multi-organización

Todo evento debe pertenecer a exactamente una organización.

Agregar al modelo existente de eventos una relación obligatoria:

```
organization = models.ForeignKey(
    Organization,
    on_delete=models.PROTECT,
    related_name="events",
)
```

Usar `PROTECT` salvo que el análisis del dominio existente demuestre una razón fuerte para otro comportamiento.

No permitir que eliminar una organización destruya accidentalmente eventos históricos.

6.1 Estados de publicación

El modelo debe distinguir inequívocamente al menos:

```
draft
published
```

Reutilizar estados existentes si ya representan correctamente esta semántica.

Opcionalmente conservar o soportar:

```
cancelled
archived
```

No introducir nuevos estados redundantes si el modelo actual ya resuelve esos casos.

6.2 Visibilidad pública

Un evento solo debe ser público si:

```
event.status == published
AND
event.organization.is_active == True
```

Adaptar la comprobación a los campos reales.

Los eventos borrador nunca deben ser accesibles públicamente por conocer su URL.

Esto debe verificarse en backend.

6.3 Slugs

Preferencia:

```
organization slug + event slug
```

La unicidad del slug del evento puede ser por organización:

```
UniqueConstraint(
    fields=["organization", "slug"],
    name="unique_event_slug_per_organization",
)
```

Antes de cambiar restricciones existentes, analizar compatibilidad con URLs y datos actuales.

No romper URLs públicas existentes sin necesidad.

7. Migración de datos existentes

La introducción de `Organization` debe realizarse sin perder datos.

7.1 Organización inicial

Crear una migración de datos que genere una organización inicial para los datos existentes.

Por ejemplo:

Fuego Austral

Usar el nombre y slug correctos según el contexto actual del proyecto.

7.2 Asignación de eventos existentes

Todos los eventos existentes deben asignarse a la organización inicial.

Secuencia recomendada:

1. crear `Organization` ;
2. crear `OrganizationMembership` ;
3. agregar `Event.organization` temporalmente nullable;
4. crear organización inicial mediante migración de datos;
5. asignar todos los eventos existentes;
6. verificar que no queden eventos sin organización;
7. convertir `Event.organization` en obligatorio.

No agregar directamente un FK non-nullable con un default arbitrario persistente.

7.3 Administradores existentes

Analizar los usuarios administrativos actuales.

Crear memberships para los usuarios que deban seguir administrando la organización inicial.

No conceder roles automáticamente a todos los usuarios staff sin analizar el significado actual de esos permisos.

Documentar la regla de migración elegida.

8. Área administrativa de organizaciones

Implementar una interfaz administrativa para usuarios organizacionales fuera de la dependencia exclusiva de Django Admin.

Ruta conceptual:

```
/dashboard/
```

Estructura conceptual:

```
/dashboard/  
/dashboard/organizations/  
  
/dashboard/o/<organization-slug>/  
/dashboard/o/<organization-slug>/events/  
/dashboard/o/<organization-slug>/events/new/  
/dashboard/o/<organization-slug>/events/<event-id>/edit/
```

Los nombres exactos pueden adaptarse a la estructura existente.

8.1 Dashboard inicial

Al ingresar:

- un usuario sin memberships no debe recibir acceso administrativo;
- un usuario con una organización puede acceder a ella;
- un usuario con varias organizaciones debe poder seleccionar la organización;
- un superusuario puede conservar herramientas globales separadas.

8.2 Listado de eventos

El listado administrativo debe incluir únicamente eventos de la organización seleccionada.

Nunca hacer:

```
Event.objects.all()
```

y filtrar posteriormente en template.

Filtrar en queryset.

8.3 Crear evento

Solo usuarios autorizados pueden crear eventos.

La organización del evento:

- debe derivarse del contexto administrativo validado;
- no debe confiar en un `organization_id` libre enviado por el navegador.

Ejemplo incorrecto:

```
organization_id = request.POST["organization_id"]
```

sin validación.

Ejemplo conceptual correcto:

```
organization = get_authorized_organization(  
    request.user,  
    organization_slug,  
)  
  
event.organization = organization
```

8.4 Editar evento

La consulta debe restringirse simultáneamente por:

- identificador del evento;
- organización autorizada.

Ejemplo conceptual:

```
event = get_object_or_404(  
    Event,  
    pk=event_id,  
    organization=organization,  
)
```

Luego validar el rol necesario.

Un admin de organización A nunca puede editar un evento de B cambiando un ID o URL.

9. Catálogo público de eventos

Usuarios anónimos deben poder:

- ver un listado de eventos publicados;
- acceder al detalle de un evento publicado;

- ver información necesaria para decidir una compra;
- ver tipos de tickets disponibles.

No exigir login para explorar eventos.

9.1 Listado público

Implementar o adaptar una vista pública que muestre exclusivamente eventos publicados de organizaciones activas.

Debe excluir:

- drafts;
- eventos privados si existe esa categoría;
- eventos de organizaciones inactivas.

9.2 Detalle público

Ruta recomendada conceptualmente:

```
/o/<organization-slug>/events/<event-slug>/
```

Puede conservarse una URL existente cuando resulte necesario para compatibilidad.

El detalle debe identificar:

- título;
- organización;
- descripción;
- fecha y hora;
- ubicación si existe;
- disponibilidad;
- tipos de tickets;
- precio;
- acción de compra.

9.3 Drafts

Un draft:

- no aparece en listados públicos;
 - no es accesible por URL pública;
 - puede ser previsualizado únicamente por usuarios autorizados mediante un flujo administrativo explícito.
-

10. Usuarios públicos y autenticación

Mantener un único sistema de usuarios.

Reutilizar el modelo de usuario y `django-allauth` existentes salvo incompatibilidad demostrada.

El flujo debe permitir:

```
anónimo
↓
ve eventos
↓
selecciona evento/tickets
↓
se autentica o registra
↓
continúa compra
```

10.1 Registro

Para comprar, el usuario debe:

- iniciar sesión;
- o crear una cuenta.

Preservar el sistema de autenticación existente en la medida posible.

No crear un segundo modelo de comprador.

10.2 Usuario con doble función

Debe ser válido que:

```
User X
├─ comprador
├─ admin de Organización A
└─ editor de Organización B
```

11. Flujo de compra

El flujo objetivo es:

1. usuario ve evento publicado
2. selecciona tipo(s) de ticket y cantidad
3. se autentica si todavía es anónimo
4. se crea o confirma una orden pendiente
5. se inicia el pago
6. proveedor procesa el pago
7. backend recibe confirmación fiable
8. orden cambia a pagada
9. se emiten tickets
10. se envía email al comprador

Adaptar este flujo al sistema actual.

No reescribir el flujo de pagos si ya existe una implementación operativa.

12. Órdenes, pagos y tickets

Antes de modificar modelos, inspeccionar cuidadosamente los modelos actuales.

El dominio debe distinguir conceptualmente:

Order
OrderItem
Payment
Ticket

Sin embargo, no crear modelos duplicados si la aplicación ya posee equivalentes funcionales.

12.1 Orden

Una orden debe identificar inequívocamente:

- comprador;
- evento;
- organización correspondiente;
- estado;
- importe;
- moneda;
- fecha.

La organización puede derivarse del evento, pero considerar una FK directa si:

- simplifica aislamiento;
- simplifica reporting;
- el patrón existente lo justifica.

Si se almacena en ambos lugares:

```
order.organization
order.event.organization
```

la implementación debe garantizar consistencia.

12.2 Ticket

Cada ticket emitido debe representar una unidad individual.

Ejemplo:

```
Order 100
├─ Ticket A
├─ Ticket B
└─ Ticket C
```

Cada ticket debe tener un identificador único adecuado para:

- envío;
- validación;
- QR si el sistema actual lo utiliza;
- estado individual;
- check-in si existe.

No sustituir un sistema individual existente por un simple campo `quantity`.

13. Confirmación de pago y emisión

La emisión de tickets no debe depender exclusivamente del redirect del navegador después del pago.

La confirmación debe usar el mecanismo fiable ya disponible en la integración de pagos, especialmente el webhook cuando corresponda.

Flujo conceptual:

```
payment notification
↓
validate notification
↓
resolve payment/order
↓
confirm paid state
↓
```

```
issue missing tickets
↓
send confirmation email
```

13.1 Idempotencia obligatoria

El procesamiento debe ser idempotente.

Recibir dos veces una notificación del mismo pago no puede:

- duplicar tickets;
- duplicar órdenes;
- cobrar dos veces;
- emitir inventario duplicado.

Agregar tests específicos.

14. Email de tickets

Cuando una compra queda confirmada:

1. el usuario debe recibir un email;
2. el email debe identificar el evento;
3. debe incluir o permitir acceder a los tickets;
4. debe reutilizar el mecanismo actual siempre que sea viable.

La implementación debe contemplar errores de email sin corromper el estado de pago.

Ejemplo:

```
pago confirmado
ticket emitido
email falla
```

El ticket debe continuar existiendo.

Debe ser posible reintentar el email sin volver a emitir tickets.

15. Aislamiento multi-organización

Este requisito es crítico.

15.1 Regla general

Un usuario con permisos en organización A no adquiere acceso a B.

Debe impedirse acceso cruzado a:

- eventos;
- órdenes;
- compradores cuando se exponen administrativamente;
- reportes;
- tickets;
- caja;
- configuraciones;
- miembros.

15.2 IDOR

Agregar protección y tests contra cambios de identificadores.

Ejemplo:

Admin A puede editar:
`/dashboard/o/a/events/10/edit/`

Event 11 pertenece a B.

Cambiar manualmente `10` por `11` debe producir:

404

o:

403

según la convención elegida.

No debe mostrar ni modificar el evento.

15.3 Querysets

Toda vista administrativa tenant-scoped debe filtrar desde base de datos.

Preferir:


```
Event.objects.filter(  
    organization=authorized_organization,  
)
```

sobre:

```
Event.objects.all()
```

seguido de comprobaciones tardías.

16. Django Admin

Mantener Django Admin para administración global.

Registrar:

```
Organization  
OrganizationMembership
```

Actualizar administración de eventos para mostrar:

- organización;
- estado;
- datos relevantes existentes.

Agregar filtros por organización cuando sea útil.

El Django Admin no sustituye al dashboard organizacional.

17. Funcionalidades específicas existentes

El repositorio contiene funcionalidades históricamente específicas.

El agente debe clasificarlas en tres categorías.

Categoría A: core genérico

Mantener e integrar:

- eventos;
- autenticación;
- tickets;
- compra;

- pagos;
- emails.

Categoría B: funcionalidad genérica opcional

Mantener cuando sea viable, pero hacerla organization-aware:

- caja;
- scanner/check-in;
- cupones;
- transferencias;
- reportes.

Categoría C: funcionalidad específica de una organización

Ejemplos potenciales:

- logros específicos;
- “Mi Fuego”;
- reglas particulares de Fuego Austral;
- grupos o beneficios internos;
- ingreso anticipado específico.

No eliminar automáticamente estas funcionalidades.

Primero:

1. localizar dependencias;
2. desacoplarlas del core;
3. evitar que sean requisito para organizaciones nuevas;
4. mantener compatibilidad para la organización inicial cuando sea razonable.

Una organización genérica nueva debe poder:

```
crear evento  
publicarlo  
vender tickets  
enviar tickets
```

sin configurar funcionalidades específicas de Fuego Austral.

18. Branding específico

Eliminar del flujo genérico la dependencia obligatoria de textos y branding específicos.

Buscar referencias visibles a:

Fuego Austral
FA
Mi Fuego

Clasificarlas antes de cambiar.

No hacer un reemplazo global ciego.

18.1 Branding inicial mínimo

Agregar, cuando sea necesario para páginas públicas:

Organization.name

Opcionalmente:

Organization.logo

No implementar un sistema completo de themes en esta fase salvo que resulte trivial con la arquitectura existente.

19. Configuración de pagos

En esta primera implementación, no es obligatorio que cada organización tenga su propia cuenta de MercadoPago.

Mantener inicialmente la configuración global existente si es la opción más segura.

Sin embargo:

- no hardcodear nuevas dependencias organizacionales en el proveedor;
- estructurar el código de modo que una futura configuración por organización sea posible.

No implementar multi-account payments sin requisito explícito.

20. Inventario y concurrencia

Preservar las garantías actuales de stock.

Si los tickets tienen cupo:

- no vender por encima de capacidad;
- no introducir race conditions al adaptar el flujo;

- mantener bloqueos/transacciones existentes.

Si se modifica código de stock, agregar tests de concurrencia o transacciones cuando sea viable.

21. URLs y compatibilidad

No romper URLs existentes innecesariamente.

Cuando se introduzcan URLs organizacionales:

```
/o/<organization-slug>/...
```

evaluar:

- redirects;
- aliases;
- compatibilidad con emails antiguos;
- tickets antiguos;
- enlaces públicos existentes.

Mantener rutas antiguas cuando hacerlo sea simple y seguro.

22. Arquitectura de código recomendada

Agregar una app:

```
organizations/
```

Estructura sugerida:

```
organizations/  
├─ admin.py  
├─ apps.py  
├─ migrations/  
├─ models.py  
├─ permissions.py  
├─ urls.py  
├─ views.py  
└─ tests/
```

No mover masivamente código existente sin necesidad.

Las apps actuales pueden continuar gestionando sus dominios:

```
events/  
tickets/  
user_profile/  
caja/
```

El objetivo no es reorganizar por estética.

23. Servicios y lógica de dominio

Cuando exista lógica con efectos relevantes, evitar duplicarla en vistas.

Ejemplos:

```
create_event(...)  
publish_event(...)  
confirm_payment(...)  
issue_tickets(...)  
send_ticket_email(...)
```

Reutilizar servicios existentes cuando ya resuelvan estas operaciones.

No introducir una capa `services.py` mecánicamente para simples operaciones CRUD.

24. Migraciones Django

Todas las modificaciones de schema deben realizarse mediante migraciones Django.

Distinguir:

- schema migrations;
- data migrations.

24.1 Requisitos de migración

Las migraciones deben:

- funcionar sobre una base existente;
- preservar datos;
- no depender de imports de modelos runtime dentro de `RunPython`;
- usar `apps.get_model(...)`;
- ser reversibles cuando sea razonable;
- documentar operaciones irreversibles.

24.2 Prohibido

No:

- editar migraciones históricas ya aplicadas para simular el nuevo estado;
 - borrar tablas manualmente;
 - exigir reset de base de datos;
 - asumir una base vacía.
-

25. Tests obligatorios

Agregar tests automatizados.

La implementación no se considera terminada sin tests del aislamiento organizacional.

25.1 Organización

Probar:

- creación;
- slug único;
- membership única por usuario/organización;
- usuario con memberships múltiples.

25.2 Permisos

Probar:

```
owner A puede crear evento A
admin A puede editar evento A
editor A puede editar evento A
usuario común no puede editar evento A
admin A no puede editar evento B
admin A no puede crear evento para B
```

25.3 Eventos públicos

Probar:

```
published A -> visible anónimamente
draft A -> no visible
published de org inactiva -> no visible
```

25.4 Compra

Probar el camino principal:

```
public event
→ authenticated buyer
→ order
→ payment confirmed
→ tickets issued
→ email scheduled/sent
```

Adaptar al mecanismo actual.

25.5 Idempotencia

Probar:

```
same payment confirmation x2
```

Resultado:

```
one logical payment confirmation
one set of tickets
no duplicates
```

25.6 Acceso cruzado

Agregar tests específicos contra IDOR:

```
admin org A requests event org B by known ID
```

Resultado esperado:

```
403 or 404
```

Nunca `200`.

26. Criterios de aceptación funcional

La tarea se considera completa cuando se cumplen todos los siguientes criterios.

AC-01 — múltiples organizaciones

Dadas dos organizaciones:

```
Organization A
Organization B
```

ambas pueden coexistir en la misma base de datos.

AC-02 — membresías

Un usuario puede ser:

```
admin de A
editor de B
```

sin conflicto.

AC-03 — creación aislada

Un admin de A puede crear eventos para A.

No puede crear eventos para B sin membership apropiada.

AC-04 — edición aislada

Un admin de A puede editar eventos de A.

No puede editar eventos de B incluso conociendo su ID.

AC-05 — publicación

Un usuario autorizado puede publicar un evento.

AC-06 — catálogo anónimo

Un usuario sin login puede listar y abrir eventos publicados.

AC-07 — drafts privados

Un evento draft no es accesible públicamente.

AC-08 — registro para compra

Un usuario público puede crear cuenta o iniciar sesión para completar una compra.

AC-09 — compra

Una compra válida produce una orden según el dominio existente.

AC-10 — pago

Una confirmación válida del proveedor actualiza la compra correctamente.

AC-11 — tickets

Una compra pagada genera los tickets correspondientes.

AC-12 — email

Los tickets se envían por email mediante el mecanismo existente adaptado.

AC-13 — idempotencia

Procesar repetidamente la misma confirmación no duplica tickets.

AC-14 — compatibilidad

Los datos históricos existentes permanecen accesibles.

AC-15 — organización inicial

Los eventos existentes quedan asociados a una organización inicial mediante migración.

AC-16 — tests

La suite nueva y existente pasa.

27. Requisitos no funcionales

Seguridad

- validar permisos en backend;
- evitar acceso cross-tenant;
- no confiar en IDs enviados por formularios;
- preservar validación de webhooks;
- no registrar secretos.

Rendimiento

Agregar índices cuando los nuevos filtros frecuentes lo justifiquen, especialmente para relaciones como:

```
Event.organization
OrganizationMembership.organization
OrganizationMembership.user
```

No hacer optimización especulativa excesiva.

Auditoría

Preservar mecanismos de auditoría existentes.

Cuando ya exista `django-auditlog` u otro mecanismo, registrar adecuadamente nuevos modelos importantes si coincide con el patrón del proyecto.

Logging

Agregar logging útil en:

- errores de migración lógica;
- confirmaciones de pago;
- emisión de tickets;
- fallos de email.

No registrar:

- tokens;
- secretos;
- credenciales;
- información de pago sensible.

28. Fases de implementación

Implementar en el siguiente orden.

Fase 1 — análisis y baseline

1. ejecutar tests existentes;
2. documentar fallos preexistentes;
3. inspeccionar modelos;
4. inspeccionar flujos;
5. identificar dependencias específicas.

No corregir problemas no relacionados salvo que bloqueen la tarea.

Fase 2 — organizaciones

Implementar:

```
Organization
OrganizationMembership
```

Agregar:

- admin;
- constraints;
- tests.

Fase 3 — migración de eventos

Agregar:

```
Event.organization
```

Crear:

- organización inicial;
- data migration;
- asignación de eventos;
- constraint final.

Fase 4 — permisos

Implementar helpers centralizados.

Agregar tests cross-organization antes de construir toda la UI.

Fase 5 — dashboard organizacional

Implementar:

- selección de organización;
- listado de eventos;
- creación;
- edición;
- publicación según rol.

Fase 6 — catálogo público

Adaptar:

- listado;
- detalle;
- visibilidad;
- drafts.

Fase 7 — flujo de compra

Asegurar:

- login/registro;
- continuidad de compra;
- orden;
- pago.

Fase 8 — emisión y email

Asegurar:

- confirmación fiable;
- idempotencia;
- tickets;
- email;
- reintentos seguros.

Fase 9 — desacoplamiento específico

Eliminar dependencias obligatorias de funcionalidades específicas para nuevas organizaciones.

No ampliar alcance más de lo necesario.

Fase 10 — regresión

Ejecutar:

- tests completos;
- comprobaciones de migraciones;
- revisión de permisos;
- revisión manual de camino principal.

29. Forma de trabajo del agente

El agente debe trabajar de manera autónoma sobre el repositorio.

Antes de cada cambio importante

Inspeccionar primero el código relacionado.

No crear implementaciones paralelas sin comprobar si ya existe funcionalidad equivalente.

Después de cada fase

Ejecutar:

```
python manage.py check
```

Ejecutar tests relevantes.

Ejecutar también la suite completa periódicamente.

Migraciones

Verificar:

```
python manage.py makemigrations --check
```

cuando corresponda.

Comprobar el plan:

```
python manage.py showmigrations
```

y, cuando sea útil:

```
python manage.py migrate --plan
```

Calidad

Usar las herramientas de linting/formato existentes en el repositorio.

No introducir herramientas nuevas salvo necesidad real.

30. Reglas explícitas para evitar cambios destructivos

El agente NO debe:

1. reescribir la aplicación desde cero;
2. cambiar de Django a otro framework;
3. crear una SPA;
4. reemplazar PostgreSQL;
5. reemplazar MercadoPago sin necesidad;
6. eliminar funcionalidades existentes solo porque sean específicas;
7. borrar migraciones históricas;
8. resetear la base de datos;
9. cambiar el modelo de usuario sin una necesidad demostrada;
10. duplicar el sistema de usuarios;
11. introducir un tenant framework externo sin necesidad;
12. implementar un schema PostgreSQL por organización;
13. crear una base por organización;
14. modificar CI/CD salvo que sea necesario para los cambios;
15. hacer refactors masivos no relacionados;
16. cambiar estilos visuales de toda la aplicación como parte de esta tarea;
17. asumir que ocultar una acción en frontend constituye autorización.

31. Decisiones que el agente puede adaptar

El agente puede adaptar:

- nombres exactos de modelos;
- ubicación exacta de helpers;
- class-based vs function-based views;
- nombres de URLs;
- estructura precisa de templates;
- uso de forms existentes;
- mecanismos de servicios existentes.

Solo cuando:

1. respete los requisitos funcionales;
2. mantenga aislamiento organizacional;
3. preserve compatibilidad;
4. reduzca riesgo;
5. se ajuste mejor al código real.

Documentar brevemente cualquier desviación importante respecto de esta especificación.

32. Entregables esperados

Al finalizar, entregar:

32.1 Código

Todos los cambios necesarios.

32.2 Migraciones

Schema y data migrations.

32.3 Tests

Tests de:

- organizaciones;
- memberships;
- permisos;
- acceso cruzado;
- eventos públicos;
- compra;
- idempotencia.

32.4 Resumen técnico

Crear un resumen con:

- modelos agregados
- modelos modificados
- migraciones
- endpoints/vistas agregados
- permisos
- flujo de compra afectado
- flujo de email afectado
- compatibilidad
- decisiones técnicas
- limitaciones pendientes

32.5 Instrucciones de verificación manual

Incluir pasos concretos para comprobar:

1. crear Organización A
2. crear Organización B
3. asignar admin A

4. asignar admin B
5. crear eventos
6. verificar aislamiento
7. publicar
8. acceder anónimamente
9. registrar comprador
10. completar compra de prueba
11. confirmar tickets
12. confirmar email

33. Definición de terminado

La implementación está terminada únicamente cuando:

- existen múltiples organizaciones;
- los permisos son organization-scoped;
- administradores pueden gestionar solo sus eventos;
- eventos publicados son públicos sin login;
- drafts permanecen privados;
- compradores pueden registrarse;
- compradores pueden completar el flujo de compra;
- una compra confirmada genera tickets;
- los tickets se envían por email;
- el procesamiento es idempotente;
- eventos existentes fueron migrados;
- no hay pérdida conocida de datos;
- los tests de aislamiento pasan;
- los tests existentes relevantes continúan pasando;
- el agente entrega un resumen de cambios y limitaciones.

34. Instrucción final al agente

Comienza inspeccionando el repositorio real y comparando esta especificación con los modelos y flujos existentes.

No implementes ciegamente los pseudomodelos de este documento. Son contratos conceptuales.

Para cada capacidad:

1. identifica primero la implementación existente;
2. reutilízala cuando sea adecuada;
3. extiéndela para soportar organizaciones;
4. agrega aislamiento de permisos;
5. preserva datos históricos;
6. agrega tests;
7. evita cambios no relacionados.

Prioridad principal:

```
correctitud
> aislamiento organizacional
> preservación de datos
> compatibilidad
> simplicidad
> refactor estético
```

Ante una ambigüedad técnica, elegir la solución menos destructiva que satisfaga los criterios de aceptación y documentar la decisión.